

(1) Cell-01:

Input:

```
!pip install ultralytics supervision -q

import torch
from pathlib import Path
print(f"PyTorch: {torch.__version__}")
print(f"CUDA: {torch.cuda.is_available()}")
for i in range(torch.cuda.device_count()):
    print(f" GPU {i}: {torch.cuda.get_device_name(i)}")
```

Output:

```
PyTorch: 2.10.0+cu128
CUDA: True
 GPU 0: Tesla T4
 GPU 1: Tesla T4
```

(2) Cell-02:

Input:

```
from pathlib import Path

BASE = Path('/kaggle/input/datasets/banuprasadb/visdrone-dataset')
VISDRONE = BASE / 'VisDrone_Dataset'
for item in sorted(VISDRONE.rglob('*')):
    depth = len(item.relative_to(VISDRONE).parts)
    if depth <= 3:
        print(' ' * depth + item.name + ('/' if item.is_dir() else ''))
```

Output:

```
VisDrone2019-DET-test-challenge/
images/
 0000000_00098_d_0000001.jpg
 0000000_01013_d_0000003.jpg
 0000000_01619_d_0000004.jpg
 0000000_02021_d_0000005.jpg
 0000000_02309_d_0000006.jpg
 0000000_02652_d_0000007.jpg
 0000000_03019_d_0000008.jpg
 0000005_00509_d_0000002.jpg
 0000009_00009_d_0000001.jpg
 0000009_00811_d_0000003.jpg
 0000009_01339_d_0000005.jpg
 0000009_01723_d_0000006.jpg
 0000009_01947_d_0000007.jpg
 0000009_03179_d_0000010.jpg
 0000009_03611_d_0000011.jpg
 0000009_03907_d_0000012.jpg
```

(3) Cell-03:

Input:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from collections import defaultdict
from pathlib import Path

BASE = Path('/kaggle/input/datasets/banuprasadb/visdrone-dataset')
VISDRONE = BASE / 'VisDrone_Dataset'
TRAIN_IMG = VISDRONE / 'VisDrone2019-DET-train' / 'images'
TRAIN_ANN = VISDRONE / 'VisDrone2019-DET-train' / 'labels'
VAL_IMG = VISDRONE / 'VisDrone2019-DET-val' / 'images'
```

```

VAL_ANN = VISDRONE / 'VisDrone2019-DET-val' / 'labels'
CLASSES = ['pedestrian', 'people', 'bicycle', 'car', 'van',
            'truck', 'tricycle', 'awning-tricycle', 'bus', 'motor']

# — Class distribution & bbox stats —————
class_counts = defaultdict(int)
bbox_areas, aspect_ratios = [], []
ann_files = sorted(TRAIN_ANN.glob('*.*txt'))
print(f"Total training annotation files: {len(ann_files)}")
for ann_path in ann_files[:500]:
    img_path = TRAIN_IMG / (ann_path.stem + '.jpg')
    if not img_path.exists():
        continue
    img = cv2.imread(str(img_path))
    if img is None:
        continue
    H, W = img.shape[:2]
    with open(ann_path) as f:
        for line in f:
            parts = line.strip().split()
            if len(parts) != 5:
                continue
            cls = int(parts[0])
            cx, cy, nw, nh = float(parts[1]), float(parts[2]), float(parts[3]), float(parts[4])
            class_counts[cls] += 1
            bw = nw * W
            bh = nh * H
            if bw > 0 and bh > 0:
                bbox_areas.append(bw * bh)
                aspect_ratios.append(bw / bh)
print("\nClass distribution (500-image sample):")
for cid, count in sorted(class_counts.items()):
    print(f" [{cid}] {CLASSES[cid]:20s}: {count:6d}")

# — Figure 1: Stats —————
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle('VisDrone Dataset Analysis', fontsize=14, fontweight='bold')
labels = [CLASSES[k] for k in sorted(class_counts)]
values = [class_counts[k] for k in sorted(class_counts)]
axes[0].barh(labels, values, color='steelblue')
axes[0].set_title('Class Distribution (500 images)')
axes[0].set_xlabel('Count')
axes[1].hist(bbox_areas, bins=50, color='coral', edgecolor='black')
axes[1].set_title('Bounding Box Area Distribution')
axes[1].set_xlabel('Area (px²)')
axes[1].set_yscale('log')
axes[2].hist(aspect_ratios, bins=50, color='green', edgecolor='black', alpha=0.7)
axes[2].set_title('Aspect Ratio Distribution')
axes[2].set_xlabel('W/H')
plt.tight_layout()
plt.savefig('/kaggle/working/dataset_analysis.png', dpi=150)
plt.show()

# — Figure 2: Sample annotated images —————
COLOR_MAP = {
    0: (1.0, 0.3, 0.3), # pedestrian - red
    1: (1.0, 0.6, 0.0), # people     - orange
    3: (0.0, 0.8, 1.0), # car       - cyan
}

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle('Sample Annotated Images (red=pedestrian, orange=people, cyan=car)',
             fontsize=12, fontweight='bold')
for ax, ann_path in zip(axes.flatten(), ann_files[:6]):
    img_path = TRAIN_IMG / (ann_path.stem + '.jpg')
    img = cv2.imread(str(img_path))
    if img is None:
        ax.axis('off')
        continue
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    H, W = img.shape[:2]

```

```

ax.imshow(img_rgb)
with open(ann_path) as f:
    for line in f:
        parts = line.strip().split()
        if len(parts) != 5:
            continue
        cls = int(parts[0])
        cx, cy, nw, nh = float(parts[1]), float(parts[2]), float(parts[3]), float(parts[4])
        bx = (cx - nw / 2) * W
        by = (cy - nh / 2) * H
        bw = nw * W
        bh = nh * H
        color = COLOR_MAP.get(cls, (0.7, 0.7, 0.7))
        rect = patches.Rectangle((bx, by), bw, bh,
                                linewidth=1, edgecolor=color, facecolor='none')

        ax.add_patch(rect)
ax.set_title(f"{ann_path.stem[:22]}\n{W}x{H}", fontsize=8)
ax.axis('off')
plt.tight_layout()
plt.savefig('/kaggle/working/sample_annotations.png', dpi=150)
plt.show()
print(f"\n✓ Median bbox area: {np.median(bbox_areas):.1f} px²")
print(f"✓ Visualizations saved")

```

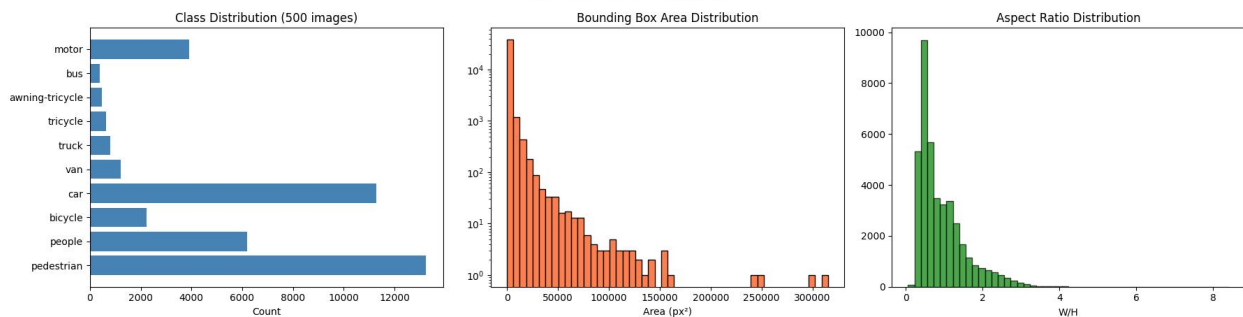
Output:

Total training annotation files: 6471

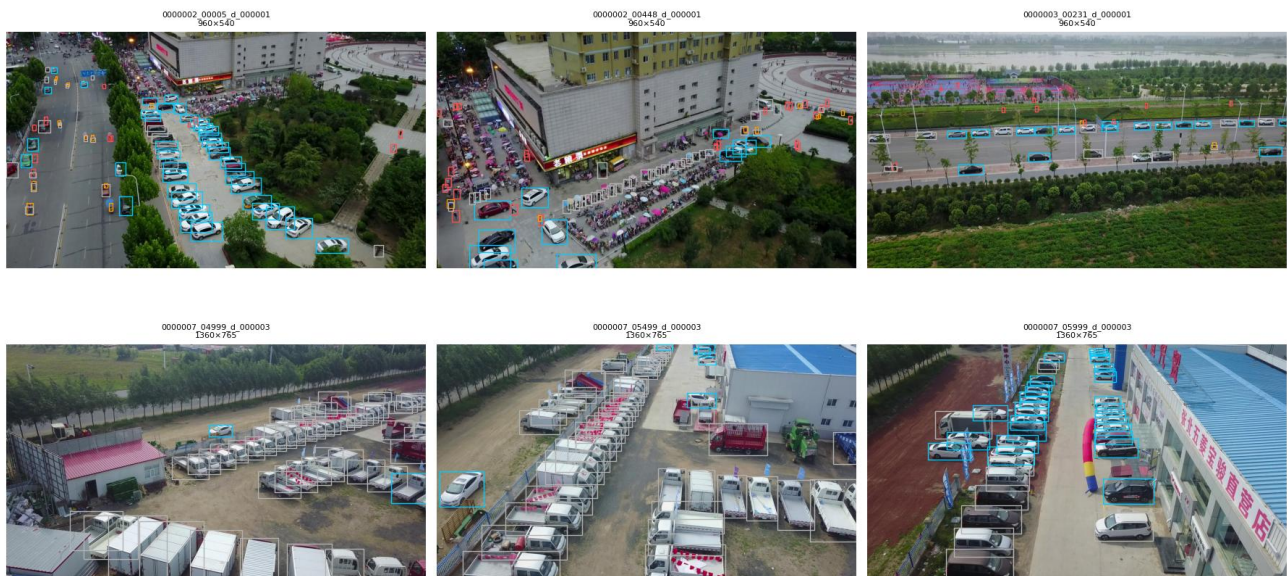
Class distribution (500-image sample):

[0] pedestrian	: 13245
[1] people	: 6185
[2] bicycle	: 2245
[3] car	: 11291
[4] van	: 1209
[5] truck	: 807
[6] tricycle	: 645
[7] awning-tricycle	: 469
[8] bus	: 377
[9] motor	: 3923

VisDrone Dataset Analysis



Sample Annotated Images (red=pedestrian, orange=people, cyan=car)



✓ Median bbox area: 520.0 px²
✓ Visualizations saved

(4) Cell-04:

Input:

```
from pathlib import Path

VISDRONE = Path('/kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_Dataset')
yaml_content = f"""path: {VISDRONE}
train: VisDrone2019-DET-train/images
val: VisDrone2019-DET-val/images
nc: 10
names: ['pedestrian', 'people', 'bicycle', 'car', 'van',
        'truck', 'tricycle', 'awning-tricycle', 'bus', 'motor']
"""

YAML_PATH = Path('/kaggle/working/visdrone.yaml')
with open(YAML_PATH, 'w') as f:
    f.write(yaml_content)
print(f"✓ YAML written to {YAML_PATH}")
print(yaml_content)
```

Output:

```
✓ YAML written to /kaggle/working/visdrone.yaml
path: /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_Dataset
train: VisDrone2019-DET-train/images
val: VisDrone2019-DET-val/images

nc: 10
names: ['pedestrian', 'people', 'bicycle', 'car', 'van',
        'truck', 'tricycle', 'awning-tricycle', 'bus', 'motor']
```

(5) Cell-05:

Input:

```
from ultralytics import YOLO
from pathlib import Path

YAML_PATH = Path('/kaggle/working/visdrone.yaml')
model = YOLO('yolov8s.pt')
results = model.train(
    data = str(YAML_PATH),
    epochs = 20,
    imgsz = 640,
    batch = 32,
    device = '0,1',
    workers = 4,
    project = '/kaggle/working/runs',
    name = 'visdrone_yolov8s',
    # Augmentation
    hsv_h = 0.015,
    hsv_s = 0.7,
    hsv_v = 0.4,
    flipud = 0.5,
    fliplr = 0.5,
    mosaic = 1.0,
    scale = 0.5,
    save_period = 5,
    val = True,
    plots = True,
)
BEST_MODEL = '/kaggle/working/runs/visdrone_yolov8s/weights/best.pt'
print(f"\n✓ Training complete")
```

```
print(f" Best model: {BEST_MODEL}")
```

Output:

```
Creating new Ultralytics Settings v0.0.6 file 
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key-value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
Ultralytics 8.4.51 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
CUDA:1 (Tesla T4, 14913MiB)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugument, batch=32, bgr=0.0, box=7.5, cache=False, cfg=None, classes=None, close_mosaic=10, cls
Downloading https://ultralytics.com/assets/Arial.ttf to '/root/.config/Ultralytics/Arial.ttf': 100%  755.1KB 17.4MB/s 0.0s
Overriding model.yaml nc=80 with nc=10

   from  n  params module                                arguments
   --  --  --  --  --
0       -1 1     928  ultralytics.nn.modules.conv.Conv          [3, 32, 3, 2]
1       -1 1    18560 ultralytics.nn.modules.conv.Conv          [32, 64, 3, 2]
2       -1 1    29056 ultralytics.nn.modules.block.C2f          [64, 64, 1, True]
3       -1 1    73984 ultralytics.nn.modules.conv.Conv          [64, 128, 3, 2]
4       -1 2   197632 ultralytics.nn.modules.block.C2f          [128, 128, 2, True]
5       -1 1   295424 ultralytics.nn.modules.conv.Conv          [128, 256, 3, 2]
6       -1 2   788480 ultralytics.nn.modules.block.C2f          [256, 256, 2, True]
7       -1 1  1180672 ultralytics.nn.modules.conv.Conv          [256, 512, 3, 2]
8       -1 1  1838080 ultralytics.nn.modules.block.C2f          [512, 512, 1, True]
9       -1 1   656896 ultralytics.nn.modules.block.SPPF          [512, 512, 5]
10      -1 1         0 torch.nn.modules.upsampling.Upsample      [None, 2, 'nearest']
11      [-1, 6] 1         0 ultralytics.nn.modules.conv.Concat          [1]
12      -1 1   591360 ultralytics.nn.modules.block.C2f          [768, 256, 1]
13      -1 1         0 torch.nn.modules.upsampling.Upsample      [None, 2, 'nearest']
14     [-1, 4] 1         0 ultralytics.nn.modules.conv.Concat          [1]
...

Results saved to /kaggle/working/runs/visdrone_yolov8s-3

✓ Training complete
Best model: /kaggle/working/runs/visdrone_yolov8s/weights/best.pt
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

(6) Cell-06:

Input:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
from ultralytics import YOLO

BASE      = Path('/kaggle/input/datasets/banuprasadb/visdrone-dataset')
VISDRONE  = BASE / 'VisDrone_Dataset'
VAL_IMG   = VISDRONE / 'VisDrone2019-DET-val' / 'images'
BEST_MODEL = '/kaggle/working/runs/visdrone_yolov8s-2/weights/best.pt'
model = YOLO(BEST_MODEL)
PERSON_CLASSES = {0, 1}
CAR_CLASSES    = {3}

def detect_and_annotate(img_path, model, conf=0.25):
    img = cv2.imread(str(img_path))
    results = model(img, conf=conf, verbose=False)[0]
    person_count = 0
    car_count    = 0
    for box in results.bboxes:
        cls = int(box.cls)
        if cls in PERSON_CLASSES:
            person_count += 1
        elif cls in CAR_CLASSES:
            car_count += 1
    annotated = results.plot()
    banner = np.zeros((55, annotated.shape[1], 3), dtype=np.uint8)
    banner[:] = (25, 25, 25)
    cv2.putText(banner, f'Persons: {person_count}',
                (20, 36), cv2.FONT_HERSHEY_DUPLEX, 0.9, (80, 255, 120), 2)
    cv2.putText(banner, f'Cars: {car_count}',
                (annotated.shape[1] // 2, 36),
                cv2.FONT_HERSHEY_DUPLEX, 0.9, (0, 200, 255), 2)
    out = np.vstack([banner, annotated])
    return out, person_count, car_count

VAL_SAMPLES = sorted(VAL_IMG.glob('*.*jpg'))[:12]
OUT_DIR = Path('/kaggle/working/detections')
OUT_DIR.mkdir(exist_ok=True)
fig, axes = plt.subplots(3, 4, figsize=(20, 14))
fig.suptitle('Human & Car Detection with Counting', fontsize=15, fontweight='bold')
total_persons, total_cars = 0, 0
for ax, img_path in zip(axes.flatten(), VAL_SAMPLES):
    annotated, p, c = detect_and_annotate(img_path, model)
    total_persons += p
```



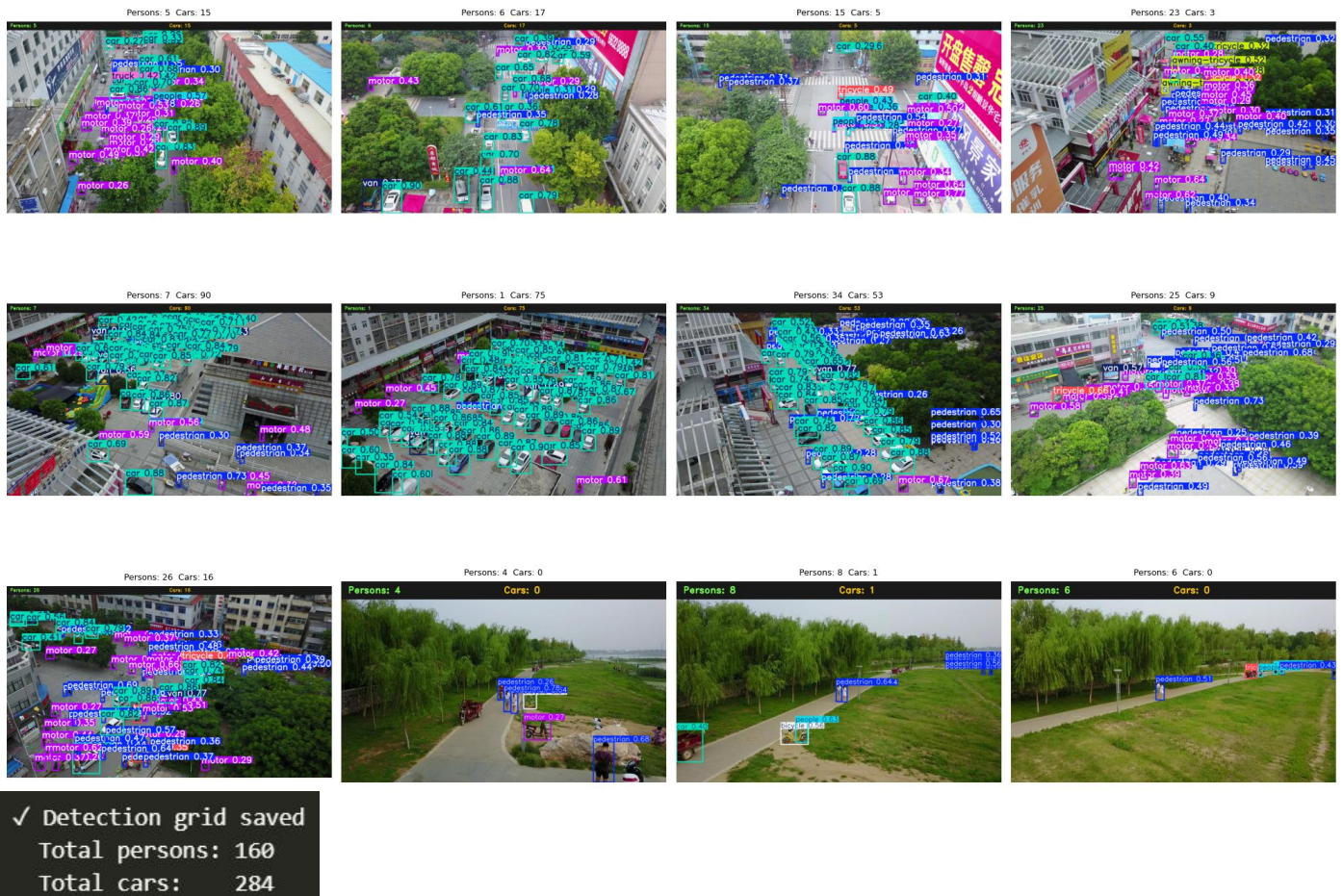
```

total_cars    += c
cv2.imwrite(str(OUT_DIR / img_path.name), annotated)
ax.imshow(cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB))
ax.set_title(f'Persons: {p}   Cars: {c}', fontsize=9)
ax.axis('off')
plt.tight_layout()
plt.savefig('/kaggle/working/detection_grid.png', dpi=120)
plt.show()
print(f"✓ Detection grid saved")
print(f"  Total persons: {total_persons}")
print(f"  Total cars:      {total_cars}")

```

Output:

Human & Car Detection with Counting



(7) Cell-07:

Input:

```

import cv2
import numpy as np
from pathlib import Path
from ultralytics import YOLO

BASE      = Path('/kaggle/input/datasets/banuprasadb/visdrone-dataset')
VISDRONE  = BASE / 'VisDrone_Dataset'
VAL_IMG   = VISDRONE / 'VisDrone2019-DET-val' / 'images'
BEST_MODEL = '/kaggle/working/runs/visdrone_yolov8s-2/weights/best.pt'
model = YOLO(BEST_MODEL)
PERSON_CLASSES = {0, 1}
CAR_CLASSES    = {3}
val_imgs      = sorted(VAL_IMG.glob('*.jpg'))[:60]
TRACK_OUT     = Path('/kaggle/working/tracking_output.mp4')
BANNER_H      = 55
# — Fix: use a consistent output size for the video writer —————
OUT_W, OUT_H = 1280, 720 # resize all frames to this

```

```

fourcc = cv2.VideoWriter_fourcc(*'mp4v')
writer = cv2.VideoWriter(str(TRACK_OUT), fourcc, 5, (OUT_W, OUT_H + BANNER_H))
for img_path in val_imgs:
    img = cv2.imread(str(img_path))
    if img is None:
        continue
    # Resize to consistent size before inference
    img = cv2.resize(img, (OUT_W, OUT_H))
    results = model.track(img, persist=True, tracker='bytetrack.yaml',
                          conf=0.25, verbose=False)[0]

    person_count = 0
    car_count = 0
    if results.bboxes.id is not None:
        for box, track_id, cls in zip(results.bboxes.xyxy,
                                      results.bboxes.id.int(),
                                      results.bboxes.cls.int()):
            x1, y1, x2, y2 = map(int, box.tolist())
            tid = int(track_id)
            cls = int(cls)
            if cls in PERSON_CLASSES:
                color = (80, 255, 120)
                label = f"P#{tid}"
                person_count += 1
            elif cls in CAR_CLASSES:
                color = (0, 200, 255)
                label = f"C#{tid}"
                car_count += 1
            else:
                continue
            cv2.rectangle(img, (x1, y1), (x2, y2), color, 2)
            cv2.putText(img, label, (x1, max(y1 - 5, 10)),
                       cv2.FONT_HERSHEY_SIMPLEX, 0.45, color, 1)
    # Banner uses current frame width (always OUT_W now)
    banner = np.zeros((BANNER_H, OUT_W, 3), dtype=np.uint8)
    banner[:] = (25, 25, 25)
    cv2.putText(banner,
                f'ByteTrack | Persons: {person_count} Cars: {car_count}',
                (15, 36), cv2.FONT_HERSHEY_DUPLEX, 0.85, (255, 255, 255), 2)
    frame = np.vstack([banner, img])
    writer.write(frame)
writer.release()
print(f"✓ Tracking video saved → {TRACK_OUT}")

```

Output:

```

requirements: Ultralytics requirement ['lap>=0.5.12'] not found, attempting AutoUpdate...
Using Python 3.12.12 environment at: /usr
Resolved 2 packages in 240ms
Prepared 1 package in 78ms
Installed 1 package in 3ms
+ lap==0.5.13

requirements: AutoUpdate success ✅ 0.8s
WARNING ⚠ requirements: Restart runtime or rerun command for updates to take effect

✓ Tracking video saved → /kaggle/working/tracking_output.mp4

```

(8) Cell-08:

Input:

```

import matplotlib.pyplot as plt
import matplotlib.image as mpimg
from pathlib import Path
from ultralytics import YOLO

BEST_MODEL = '/kaggle/working/runs/visdrone_yolov8s-2/weights/best.pt'
RUN_DIR = Path('/kaggle/working/runs/visdrone_yolov8s-2')
YAML_PATH = '/kaggle/working/visdrone.yaml'

```

```

model = YOLO(BEST_MODEL)
metrics = model.val(
    data = YAML_PATH,
    conf = 0.25,
    iou = 0.5,
    verbose = True,
    plots = True,
)

print("\n== EVALUATION RESULTS ==")
print(f" mAP@0.5: {metrics.box.map50:.4f}")
print(f" mAP@0.5:0.95: {metrics.box.map:.4f}")
print(f" Precision: {metrics.box.mp:.4f}")
print(f" Recall: {metrics.box.mr:.4f}")
CLASSES = ['pedestrian', 'people', 'bicycle', 'car', 'van',
            'truck', 'tricycle', 'awning-tricycle', 'bus', 'motor']

print("\nPer-class results:")
for i, (p, r, ap50) in enumerate(zip(metrics.box.p, metrics.box.r, metrics.box.ap50)):
    print(f" {CLASSES[i]:20s} P={p:.3f} R={r:.3f} AP50={ap50:.3f}")
curve_imgs = ['results.png', 'confusion_matrix.png', 'BoxPR_curve.png', 'BoxF1_curve.png']
fig, axes = plt.subplots(2, 2, figsize=(18, 12))
fig.suptitle('Training Results & Evaluation', fontsize=14, fontweight='bold')
for ax, fname in zip(axes.flatten(), curve_imgs):
    p = RUN_DIR / fname
    if p.exists():
        ax.imshow(mplimg.imread(str(p)))
        ax.set_title(fname.replace('.png', '').replace('_', ' ').title())
        ax.axis('off')
plt.tight_layout()
plt.savefig('/kaggle/working/evaluation_summary.png', dpi=120)
plt.show()
print("✓ Evaluation complete")

```

Output:

```

Ultralytics 8.4.51 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Model summary (fused): 73 layers, 11,129,454 parameters, 0 gradients, 28.5 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 236.9±34.4 MB/s, size: 131.3 KB)
val: Scanning /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_Dataset/VisDrone2019-DET-val/labels... 548 images, 0 backgrounds, 0 corrupt: 100% 548/548 563.8it/s
WARNING ⚠ val: Cache directory /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone_Dataset/VisDrone2019-DET-val is not writable, cache not saved.

Class      Images  Instances  Box(P  R  mAP50  mAP50-95): 100% 35/35 3.9it/s 8.9s0.2s
-----
all         548      38759      0.58   0.298  0.248   0.152
pedestrian  520      8844       0.628  0.319  0.274   0.123
people     482      5125       0.661  0.196  0.156   0.0537
bicycle    364      1287       0.339  0.0824 0.0457  0.0195
car        515      14064      0.776  0.719  0.677   0.471
van        421      1975       0.583  0.326  0.264   0.197
truck      266      750        0.667  0.291  0.247   0.171
tricycle   337      1045       0.522  0.181  0.114   0.0689
awning-tricycle 220      532       0.345  0.132  0.0675  0.0476
bus        131      251        0.635  0.43  0.389   0.268
motor      485      4886       0.642  0.305  0.241   0.0998

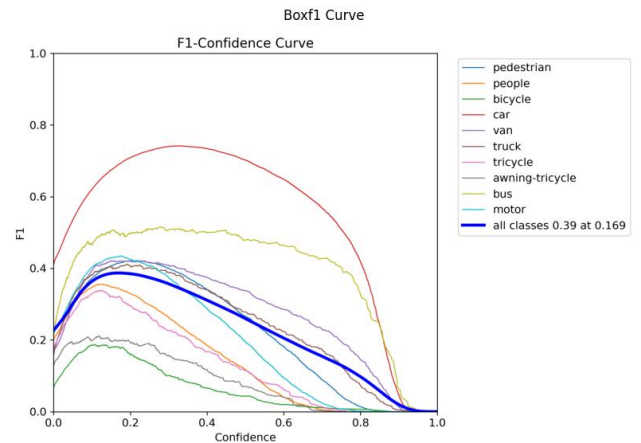
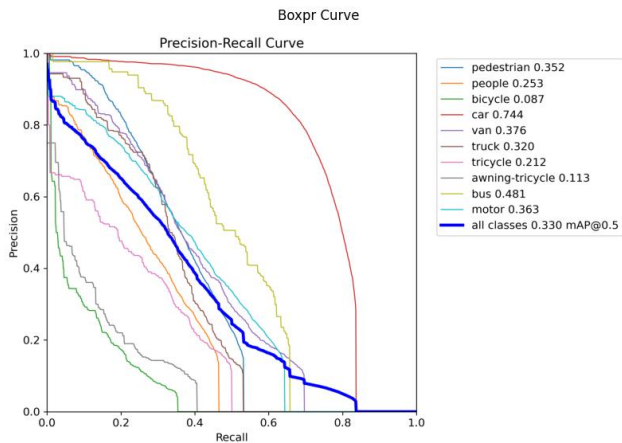
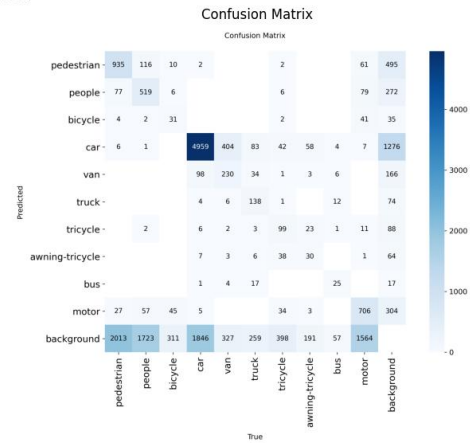
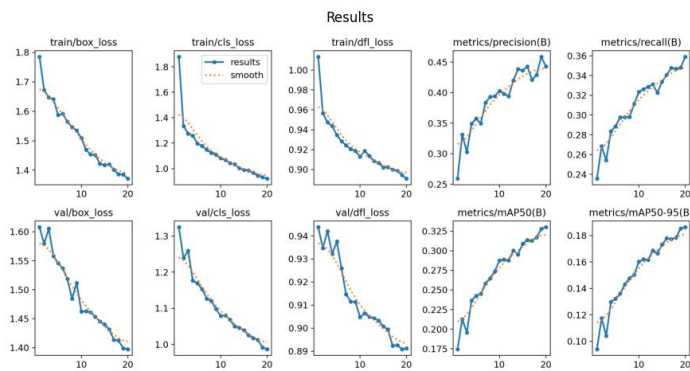
Speed: 0.5ms preprocess, 5.3ms inference, 0.0ms loss, 2.9ms postprocess per image
Results saved to /kaggle/working/runs/detect/val-5

== EVALUATION RESULTS ==
mAP@0.5:      0.2475
mAP@0.5:0.95: 0.1520
Precision:     0.5799
Recall:        0.2981
...
tricycle      P=0.522 R=0.181 AP50=0.114
awning-tricycle P=0.345 R=0.132 AP50=0.068
bus           P=0.635 R=0.430 AP50=0.389
motor         P=0.642 R=0.305 AP50=0.241

```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

Training Results & Evaluation



✓ Evaluation complete

(9) Cell-09:

Input:

```
import shutil
from pathlib import Path

FINAL = Path('/kaggle/working/final_outputs')
FINAL.mkdir(exist_ok=True)
files_to_copy = [
    '/kaggle/working/dataset_analysis.png',
    '/kaggle/working/sample_annotations.png',
    '/kaggle/working/detection_grid.png',
    '/kaggle/working/tracking_output.mp4',
    '/kaggle/working/evaluation_summary.png',
    '/kaggle/working/runs/visdrone_yolov8s-2/weights/best.pt',
]
for f in files_to_copy:
    p = Path(f)
    if p.exists():
        shutil.copy(p, FINAL / p.name)
        print(f"✓ {p.name}")
    else:
        print(f"✗ MISSING: {p.name}")
print(f"\n✓ All outputs in: {FINAL}")
```

Output:

✓ dataset_analysis.png
 ✓ sample_annotations.png
 ✓ detection_grid.png
 ✓ tracking_output.mp4
 ✓ evaluation_summary.png
 ✓ best.pt

✓ All outputs in: /kaggle/working/final_outputs

(10) Cell-10:

Input:

```
!pip install ultralytics -q

import cv2
import numpy as np
import matplotlib.pyplot as plt
import random
from pathlib import Path
from ultralytics import YOLO

BASE = Path('/kaggle/input/datasets/banuprasadb/visdrone-dataset')
VISDRONE = BASE / 'VisDrone_Dataset'
VAL_IMG = VISDRONE / 'VisDrone2019-DET-val' / 'images'
BEST_MODEL = '/kaggle/working/runs/visdrone_yolov8s-2/weights/best.pt'
model = YOLO(BEST_MODEL)
PERSON_CLASSES = {0, 1}
CAR_CLASSES = {3}
all_imgs = sorted(VAL_IMG.glob('*.jpg'))
random.seed(42)
sample_imgs = random.sample(all_imgs, 10)
fig, axes = plt.subplots(10, 2, figsize=(20, 50))
fig.suptitle('Original vs Detected (Persons=green, Cars=cyan)',
             fontsize=16, fontweight='bold', y=1.001)
for row, img_path in enumerate(sample_imgs):
    img_bgr = cv2.imread(str(img_path))
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    results = model(img_bgr, conf=0.25, verbose=False)[0]
    person_count = 0
    car_count = 0
    for box in results.boxes:
        cls = int(box.cls)
        if cls in PERSON_CLASSES:
            person_count += 1
        elif cls in CAR_CLASSES:
            car_count += 1
    annotated_bgr = results.plot()
    annotated_rgb = cv2.cvtColor(annotated_bgr, cv2.COLOR_BGR2RGB)
    banner = np.zeros((50, annotated_rgb.shape[1], 3), dtype=np.uint8)
    banner[:] = (25, 25, 25)
    cv2.putText(banner, f'Persons: {person_count}',
                (15, 33), cv2.FONT_HERSHEY_DUPLEX, 0.85, (80, 255, 120), 2)
    cv2.putText(banner, f'Cars: {car_count}',
                (annotated_rgb.shape[1] // 2, 33),
                cv2.FONT_HERSHEY_DUPLEX, 0.85, (0, 200, 255), 2)
    banner_rgb = cv2.cvtColor(banner, cv2.COLOR_BGR2RGB)
    detected_with_banner = np.vstack([banner_rgb, annotated_rgb])
    orig_banner = np.zeros((50, img_rgb.shape[1], 3), dtype=np.uint8)
    orig_banner[:] = (25, 25, 25)
    cv2.putText(orig_banner, 'ORIGINAL',
                (15, 33), cv2.FONT_HERSHEY_DUPLEX, 0.85, (200, 200, 200), 2)
    orig_with_banner = np.vstack([orig_banner, img_rgb])
    axes[row, 0].imshow(orig_with_banner)
    axes[row, 0].set_title(f'Original - {img_path.name[:30]}', fontsize=8)
    axes[row, 0].axis('off')
    axes[row, 1].imshow(detected_with_banner)
    axes[row, 1].set_title(f'Detected - Persons: {person_count} Cars: {car_count}', fontsize=8)
    axes[row, 1].axis('off')
plt.tight_layout()
plt.savefig('/kaggle/working/comparison_10_images.png', dpi=120, bbox_inches='tight')
plt.show()
print("✓ Comparison saved → /kaggle/working/comparison_10_images.png")
```

Output:

Original vs Detected (Persons=green, Cars=cyan)



✓ Comparison saved → /kaggle/working/comparison_10_images.png

(11) Cell-11:

Input:

```
!pip install ultralytics -q

from ultralytics import YOLO
from pathlib import Path
LAST_MODEL = '/kaggle/working/runs/visdrone_yolov8s-2/weights/last.pt'
YAML_PATH = '/kaggle/working/visdrone.yaml'
model = YOLO(LAST_MODEL)
results = model.train(
    data = YAML_PATH,
    epochs = 50,          # 30 more on top of your 20
    resume = True,
    # — Speed optimizations —————
    imgsz = 640,
    batch = 64,           # doubled from 32 — T4x2 can handle it
    device = '0,1',
    workers = 8,          # more parallel data loading
    cache = 'ram',        # biggest speedup — loads all images into RAM once
    amp = True,           # mixed precision — faster forward/backward pass

    # — Reduce overhead —————
    close_mosaic = 5,      # disable mosaic for last 5 epochs (stabilizes training)
    save_period = 10,     # save checkpoint every 10 epochs instead of 5
    val = True,
    plots = True,
    project = '/kaggle/working/runs',
    name = 'visdrone_yolov8s-2',
)
print(f"\n✓ Training complete")
print(f"  Best model: {results.save_dir}/weights/best.pt")
print(f"  Run dir:    {results.save_dir}")
```

Output:

```
WARNING ⚠️ model '/kaggle/working/runs/visdrone_yolov8s-2/weights/last.pt' is not a resumable training checkpoint (missing epoch/optimizer state). Use 'resume' only to continue incom
Ultralytics 8.4.51 🚀 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
CUDA:1 (Tesla T4, 14913MiB)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugument, batch=64, bgr=0.0, box=7.5, cache=ram, cfg=None, classes=None, close_mosaic=5, cls=0.

   from  n  params  module  arguments
0         -1 1      928  ultralytics.nn.modules.conv.Conv  [3, 32, 3, 2]
1         -1 1     18560  ultralytics.nn.modules.conv.Conv  [32, 64, 3, 2]
2         -1 1     29056  ultralytics.nn.modules.block.C2f  [64, 64, 1, True]
3         -1 1     73984  ultralytics.nn.modules.conv.Conv  [64, 128, 3, 2]
4         -1 2     197632  ultralytics.nn.modules.block.C2f  [128, 128, 2, True]
5         -1 1     295424  ultralytics.nn.modules.conv.Conv  [128, 256, 3, 2]
6         -1 2     788480  ultralytics.nn.modules.block.C2f  [256, 256, 2, True]
7         -1 1     1180672  ultralytics.nn.modules.conv.Conv  [256, 512, 3, 2]
8         -1 1     1838080  ultralytics.nn.modules.block.C2f  [512, 512, 1, True]
9         -1 1     656896  ultralytics.nn.modules.block.SPPF  [512, 512, 5]
10        -1 1         0  torch.nn.modules.upsampling.Upsample  [None, 2, 'nearest']
11        [-1, 6] 1         0  ultralytics.nn.modules.conv.Concat  [1]
12        -1 1     591360  ultralytics.nn.modules.block.C2f  [768, 256, 1]
13        -1 1         0  torch.nn.modules.upsampling.Upsample  [None, 2, 'nearest']
14        [-1, 4] 1         0  ultralytics.nn.modules.conv.Concat  [1]
15        -1 1     148224  ultralytics.nn.modules.block.C2f  [384, 128, 1]
16        -1 1     147712  ultralytics.nn.modules.conv.Conv  [128, 128, 3, 2]
17        [-1, 12] 1         0  ultralytics.nn.modules.conv.Concat  [1]
18        -1 1     493056  ultralytics.nn.modules.block.C2f  [384, 256, 1]
...
Speed: 0.1ms preprocess, 1.5ms inference, 0.0ms loss, 1.4ms postprocess per image
Results saved to /kaggle/working/runs/visdrone_yolov8s-2-4

✓ Training complete
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

(12) Cell-12:

Input:

```
from pathlib import Path
for p in sorted(Path('/kaggle/working/runs').rglob('best.pt')):
    print(p)
```

Output:

```
/kaggle/working/runs/visdrone_yolov8s-2/weights/best.pt
/kaggle/working/runs/visdrone_yolov8s-2-2/weights/best.pt
/kaggle/working/runs/visdrone_yolov8s-2-3/weights/best.pt
/kaggle/working/runs/visdrone_yolov8s-2-4/weights/best.pt
/kaggle/working/runs/visdrone_yolov8s-3/weights/best.pt
```


(13) Cell-13:

Input:

```
!pip install ultralytics -q

import cv2
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
from ultralytics import YOLO

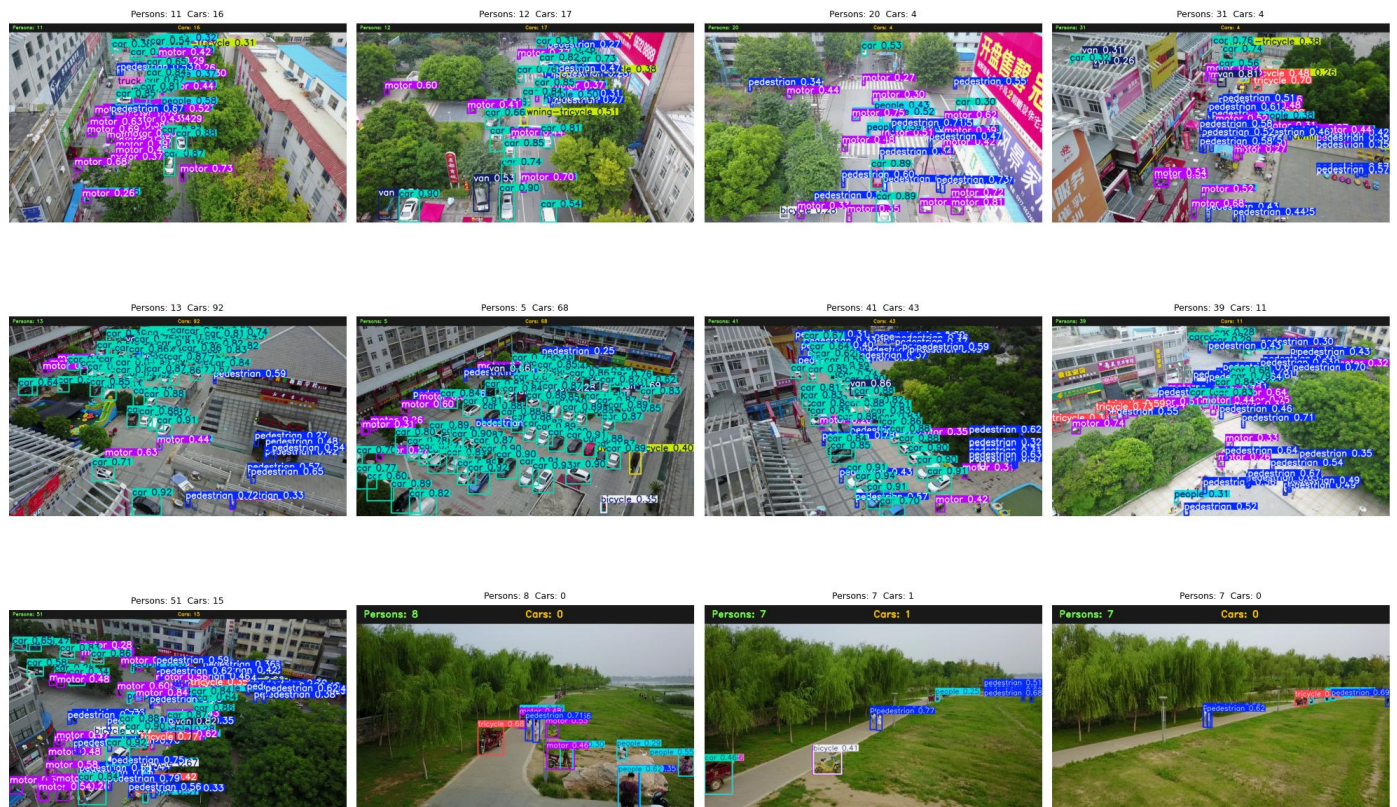
BASE = Path('/kaggle/input/datasets/banuprasadb/visdrone-dataset')
VISDRONE = BASE / 'VisDrone_Dataset'
VAL_IMG = VISDRONE / 'VisDrone2019-DET-val' / 'images'
BEST_MODEL = '/kaggle/working/runs/visdrone_yolov8s-2-3/weights/best.pt'
model = YOLO(BEST_MODEL)
PERSON_CLASSES = {0, 1}
CAR_CLASSES = {3}

def detect_and_annotate(img_path, model, conf=0.25):
    img = cv2.imread(str(img_path))
    results = model(img, conf=conf, verbose=False)[0]
    person_count = 0
    car_count = 0
    for box in results.boxes:
        cls = int(box.cls)
        if cls in PERSON_CLASSES:
            person_count += 1
        elif cls in CAR_CLASSES:
            car_count += 1
    annotated = results.plot()
    banner = np.zeros((55, annotated.shape[1], 3), dtype=np.uint8)
    banner[:] = (25, 25, 25)
    cv2.putText(banner, f'Persons: {person_count}',
                (20, 36), cv2.FONT_HERSHEY_DUPLEX, 0.9, (80, 255, 120), 2)
    cv2.putText(banner, f'Cars: {car_count}',
                (annotated.shape[1] // 2, 36),
                cv2.FONT_HERSHEY_DUPLEX, 0.9, (0, 200, 255), 2)
    out = np.vstack([banner, annotated])
    return out, person_count, car_count

VAL_SAMPLES = sorted(VAL_IMG.glob('*.*jpg'))[:12]
OUT_DIR = Path('/kaggle/working/detections')
OUT_DIR.mkdir(exist_ok=True)
fig, axes = plt.subplots(3, 4, figsize=(20, 14))
fig.suptitle('Human & Car Detection with Counting', fontsize=15, fontweight='bold')
total_persons, total_cars = 0, 0
for ax, img_path in zip(axes.flatten(), VAL_SAMPLES):
    annotated, p, c = detect_and_annotate(img_path, model)
    total_persons += p
    total_cars += c
    cv2.imwrite(str(OUT_DIR / img_path.name), annotated)
    ax.imshow(cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB))
    ax.set_title(f'Persons: {p} Cars: {c}', fontsize=9)
    ax.axis('off')
plt.tight_layout()
plt.savefig('/kaggle/working/detection_grid.png', dpi=120)
plt.show()
print(f"✓ Detection grid saved")
print(f" Total persons: {total_persons}")
print(f" Total cars: {total_cars}")
```

Output:

Human & Car Detection with Counting



```
✓ Detection grid saved
Total persons: 245
Total cars:    271
```

(14) Cell-14:

Input:

[illegible]

```

x1, y1, x2, y2 = map(int, box.tolist())
tid = int(track_id)
cls = int(cls)
if cls in PERSON_CLASSES:
    color = (80, 255, 120)
    label = f"P#{tid}"
    person_count += 1
elif cls in CAR_CLASSES:
    color = (0, 200, 255)
    label = f"C#{tid}"
    car_count += 1
else:
    continue
cv2.rectangle(img, (x1, y1), (x2, y2), color, 2)
cv2.putText(img, label, (x1, max(y1 - 5, 10)),
            cv2.FONT_HERSHEY_SIMPLEX, 0.45, color, 1)
banner = np.zeros((BANNER_H, OUT_W, 3), dtype=np.uint8)
banner[:] = (25, 25, 25)
cv2.putText(banner,
            f'ByteTrack | Persons: {person_count} Cars: {car_count}',
            (15, 36), cv2.FONT_HERSHEY_DUPLEX, 0.85, (255, 255, 255), 2)
frame = np.vstack([banner, img])
writer.write(frame)
writer.release()
print(f"✓ Tracking video saved → {TRACK_OUT}")

```

Output:

✓ Tracking video saved → /kaggle/working/tracking_output.mp4

(15) Cell-15:

Input:

```

import matplotlib.pyplot as plt
import matplotlib.image as mpimg
from pathlib import Path
from ultralytics import YOLO

BEST_MODEL = '/kaggle/working/runs/visdrone_yolov8s-2-3/weights/best.pt'
RUN_DIR = Path('/kaggle/working/runs/visdrone_yolov8s-2-3') # only needed in Cell 8
YAML_PATH = '/kaggle/working/visdrone.yaml'
model = YOLO(BEST_MODEL)
metrics = model.val(
    data = YAML_PATH,
    conf = 0.25,
    iou = 0.5,
    verbose = True,
    plots = True,
)

print("\n=== EVALUATION RESULTS ===")
print(f" mAP@0.5: {metrics.box.map50:.4f}")
print(f" mAP@0.5:0.95: {metrics.box.map:.4f}")
print(f" Precision: {metrics.box.mp:.4f}")
print(f" Recall: {metrics.box.mr:.4f}")
CLASSES = ['pedestrian', 'people', 'bicycle', 'car', 'van',
            'truck', 'tricycle', 'awning-tricycle', 'bus', 'motor']
print("\nPer-class results:")
for i, (p, r, ap50) in enumerate(zip(metrics.box.p, metrics.box.r, metrics.box.ap50)):
    print(f" {CLASSES[i]:20s} P={p:.3f} R={r:.3f} AP50={ap50:.3f}")
curve_imgs = ['results.png', 'confusion_matrix.png', 'BoxPR_curve.png', 'BoxF1_curve.png']
fig, axes = plt.subplots(2, 2, figsize=(18, 12))
fig.suptitle('Training Results & Evaluation', fontsize=14, fontweight='bold')
for ax, fname in zip(axes.flatten(), curve_imgs):
    p = RUN_DIR / fname
    if p.exists():
        ax.imshow(mpimg.imread(str(p)))
        ax.set_title(fname.replace('.png', '').replace('_', ' ').title())
    ax.axis('off')

```

```
plt.tight_layout()
plt.savefig('/kaggle/working/evaluation_summary.png', dpi=120)
plt.show()
print("✓ Evaluation complete")
```

Output:

```
Ultralytics 8.4.51 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Model summary (fused): 73 layers, 11,129,454 parameters, 0 gradients, 28.5 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 66.5±72.1 MB/s, size: 145.3 KB)
val: Scanning /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone Dataset/VisDrone2019-DET-val/labels... 548 images, 0 backgrounds, 0 corrupt: 100% 548/548 335.9
WARNING ⚠ val: Cache directory /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone Dataset/VisDrone2019-DET-val is not writable, cache not saved.
Class      Images  Instances  Box(P)      R      mAP50  mAP50-95): 100% 35/35 3.9it/s 9.0s0.2s
-----
all         548      38759      0.583      0.358  0.299  0.185
pedestrian  520      8844       0.616      0.401  0.353  0.16
people      482      5125       0.676      0.249  0.204  0.0806
bicycle     364      1287       0.351      0.129  0.0713 0.0362
car         515      14064      0.779      0.77   0.725  0.508
van         421      1975       0.572      0.399  0.321  0.239
truck       266      750        0.579      0.368  0.299  0.213
tricycle    337      1045       0.48       0.272  0.184  0.109
awning-tricycle 220     532       0.359      0.132  0.066  0.0452
bus         131      251       0.76       0.466  0.442  0.318
motor       485      4886       0.659      0.393  0.323  0.144

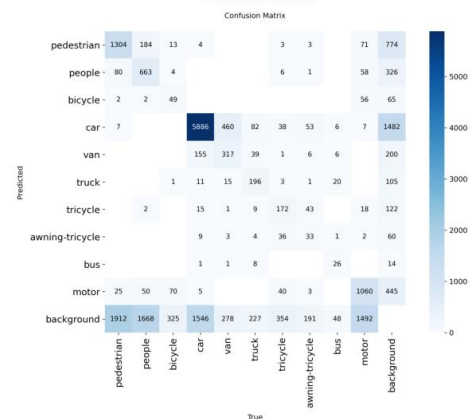
Speed: 0.6ms preprocess, 4.8ms inference, 0.0ms loss, 4.1ms postprocess per image
Results saved to /kaggle/working/runs/detect-val-6

--- EVALUATION RESULTS ---
mAP@0.5:      0.2987
mAP@0.5:0.95: 0.1853
Precision:     0.5829
Recall:       0.3578
...
tricycle      P=0.480 R=0.272 AP50=0.184
awning-tricycle P=0.359 R=0.132 AP50=0.066
bus           P=0.760 R=0.466 AP50=0.442
motor         P=0.659 R=0.393 AP50=0.323

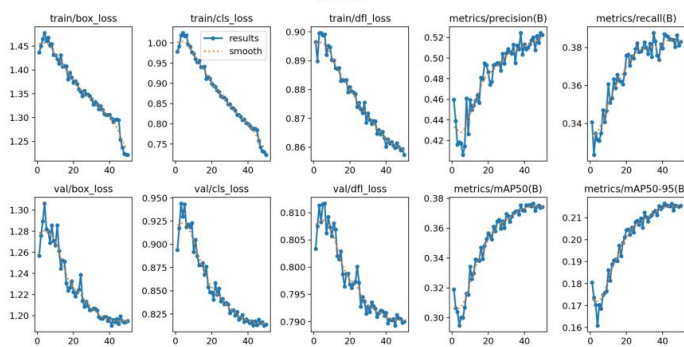
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Training Results & Evaluation

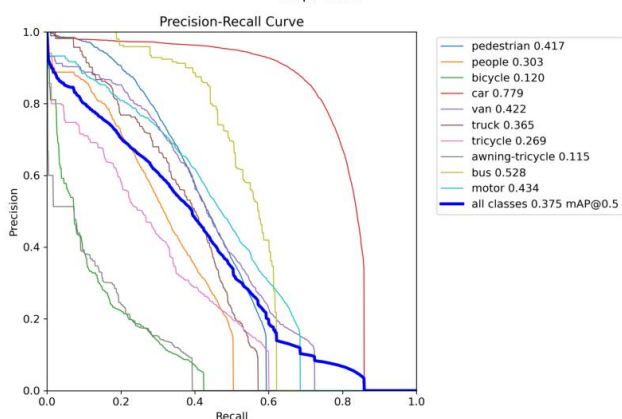
Confusion Matrix



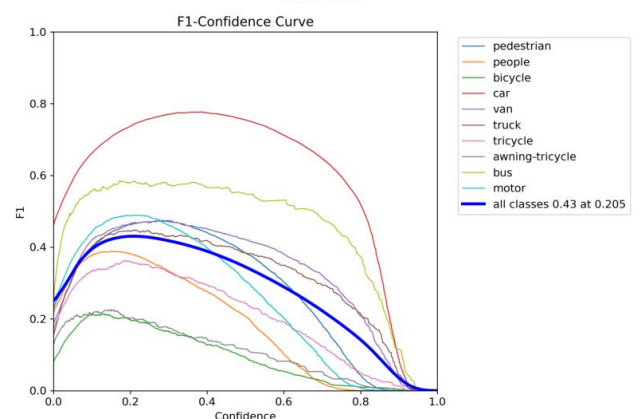
Results



Boxpr Curve



Boxf1 Curve



✓ Evaluation complete

(16) Cell-16:

Input:

```
import shutil
from pathlib import Path

FINAL = Path('/kaggle/working/final_outputs')
FINAL.mkdir(exist_ok=True)
```



```

files_to_copy = [
    '/kaggle/working/dataset_analysis.png',
    '/kaggle/working/sample_annotations.png',
    '/kaggle/working/detection_grid.png',
    '/kaggle/working/tracking_output.mp4',
    '/kaggle/working/evaluation_summary.png',
    '/kaggle/working/runs/visdrone_yolov8s-2-3/weights/best.pt',
]
for f in files_to_copy:
    p = Path(f)
    if p.exists():
        shutil.copy(p, FINAL / p.name)
        print(f"✓ {p.name}")
    else:
        print(f"✗ MISSING: {p.name}")
print(f"\n✓ All outputs in: {FINAL}")

```

Output:

```

✓ dataset_analysis.png
✓ sample_annotations.png
✓ detection_grid.png
✓ tracking_output.mp4
✓ evaluation_summary.png
✓ best.pt

✓ All outputs in: /kaggle/working/final_outputs

```

(17) Cell-17:

Input:

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
import random
from pathlib import Path
from ultralytics import YOLO

BASE      = Path('/kaggle/input/datasets/banuprasadb/visdrone-dataset')
VISDRONE  = BASE / 'VisDrone_Dataset'
VAL_IMG   = VISDRONE / 'VisDrone2019-DET-val' / 'images'
BEST_MODEL = '/kaggle/working/runs/visdrone_yolov8s-2-3/weights/best.pt'
model = YOLO(BEST_MODEL)
PERSON_CLASSES = {0, 1}
CAR_CLASSES    = {3}
all_imgs = sorted(VAL_IMG.glob('*.jpg'))
random.seed(42)
sample_imgs = random.sample(all_imgs, 10)
fig, axes = plt.subplots(10, 2, figsize=(20, 50))
fig.suptitle('Original vs Detected (Persons=green, Cars=cyan)',
             fontsize=16, fontweight='bold', y=1.001)
for row, img_path in enumerate(sample_imgs):
    img_bgr = cv2.imread(str(img_path))
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    results = model(img_bgr, conf=0.25, verbose=False)[0]
    person_count = 0
    car_count    = 0
    for box in results.boxes:
        cls = int(box.cls)
        if cls in PERSON_CLASSES:
            person_count += 1
        elif cls in CAR_CLASSES:
            car_count += 1
    annotated_bgr = results.plot()
    annotated_rgb = cv2.cvtColor(annotated_bgr, cv2.COLOR_BGR2RGB)
    banner = np.zeros((50, annotated_rgb.shape[1], 3), dtype=np.uint8)
    banner[:] = (25, 25, 25)

```

```

cv2.putText(banner, f'Persons: {person_count}',
            (15, 33), cv2.FONT_HERSHEY_DUPLEX, 0.85, (80, 255, 120), 2)
cv2.putText(banner, f'Cars: {car_count}',
            (annotated_rgb.shape[1] // 2, 33),
            cv2.FONT_HERSHEY_DUPLEX, 0.85, (0, 200, 255), 2)
banner_rgb = cv2.cvtColor(banner, cv2.COLOR_BGR2RGB)
detected_with_banner = np.vstack([banner_rgb, annotated_rgb])
orig_banner = np.zeros((50, img_rgb.shape[1], 3), dtype=np.uint8)
orig_banner[:] = (25, 25, 25)
cv2.putText(orig_banner, 'ORIGINAL',
            (15, 33), cv2.FONT_HERSHEY_DUPLEX, 0.85, (200, 200, 200), 2)
orig_with_banner = np.vstack([orig_banner, img_rgb])
axes[row, 0].imshow(orig_with_banner)
axes[row, 0].set_title(f'Original - {img_path.name[:30]}', fontsize=8)
axes[row, 0].axis('off')
axes[row, 1].imshow(detected_with_banner)
axes[row, 1].set_title(f'Detected - Persons: {person_count} Cars: {car_count}', fontsize=8)
axes[row, 1].axis('off')
plt.tight_layout()
plt.savefig('/kaggle/working/comparison_10_images.png', dpi=120, bbox_inches='tight')
plt.show()
print("✓ Comparison saved → /kaggle/working/comparison_10_images.png")

```

Output:

Original vs Detected (Persons=green, Cars=cyan)



✓ Comparison saved → /kaggle/working/comparison_10_images.png

(18) Cell-18:

Input:

```
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
from pathlib import Path
from ultralytics import YOLO

RUN_20 = Path('/kaggle/working/runs/visdrone_yolov8s-2')
RUN_50 = Path('/kaggle/working/runs/visdrone_yolov8s-2-3')
MODEL_20 = RUN_20 / 'weights/best.pt'
MODEL_50 = RUN_50 / 'weights/best.pt'
YAML_PATH = '/kaggle/working/visdrone.yaml'
# — Run validation on both models —————
print("Evaluating 20-epoch model...")
model_20 = YOLO(str(MODEL_20))
metrics_20 = model_20.val(data=YAML_PATH, conf=0.25, iou=0.5, verbose=False)
print("Evaluating 50-epoch model...")
model_50 = YOLO(str(MODEL_50))
metrics_50 = model_50.val(data=YAML_PATH, conf=0.25, iou=0.5, verbose=False)
# — Print comparison table —————
metrics_data = {
    'Metric': ['mAP@0.5', 'mAP@0.5:0.95', 'Precision', 'Recall'],
    'Epoch 20': [
        metrics_20.box.map50,
        metrics_20.box.map,
        metrics_20.box.mp,
        metrics_20.box.mr,
    ],
    'Epoch 50': [
        metrics_50.box.map50,
        metrics_50.box.map,
        metrics_50.box.mp,
        metrics_50.box.mr,
    ],
}
df = pd.DataFrame(metrics_data)
df['Improvement'] = df['Epoch 50'] - df['Epoch 20']
df['Improvement %'] = ((df['Epoch 50'] - df['Epoch 20']) / df['Epoch 20'] * 100).round(2)
df[['Epoch 20', 'Epoch 50', 'Improvement']] = df[['Epoch 20', 'Epoch 50', 'Improvement']].applymap(lambda x: f"{x:.4f}")
df['Improvement %'] = df['Improvement %'].apply(lambda x: f"{x}%" if x > 0 else f"{x}%")
print("\n== EPOCH 20 vs EPOCH 50 ==")
print(df.to_string(index=False))
# — Per-class comparison —————
CLASSES = ['pedestrian', 'people', 'bicycle', 'car', 'van',
            'truck', 'tricycle', 'awning-tricycle', 'bus', 'motor']
print("\n== PER-CLASS AP50 COMPARISON ==")
print(f" {'Class':>20s} {'Epoch 20':>10} {'Epoch 50':>10} {'Change':>10}")
print(" " + "-" * 55)
for i, cls in enumerate(CLASSES):
    ap_20 = metrics_20.box.ap50[i]
    ap_50 = metrics_50.box.ap50[i]
    change = ap_50 - ap_20
    sign = "+" if change >= 0 else ""
    print(f" {cls:>20s} {ap_20:>10.4f} {ap_50:>10.4f} {sign}{change:>10.4f}")
# — Figure 1: Bar chart comparison —————
metric_names = ['mAP@0.5', 'mAP@0.5:0.95', 'Precision', 'Recall']
vals_20 = [metrics_20.box.map50, metrics_20.box.map, metrics_20.box.mp, metrics_20.box.mr]
vals_50 = [metrics_50.box.map50, metrics_50.box.map, metrics_50.box.mp, metrics_50.box.mr]
x = range(len(metric_names))
width = 0.35
fig, axes = plt.subplots(1, 2, figsize=(18, 6))
fig.suptitle('Epoch 20 vs Epoch 50 - Model Comparison', fontsize=15, fontweight='bold')
# Bar chart
bars1 = axes[0].bar([i - width/2 for i in x], vals_20, width, label='Epoch 20', color='steelblue', alpha=0.85)
```



```

bars2 = axes[0].bar([i + width/2 for i in x], vals_50, width, Label='Epoch 50', color='coral', alpha=0.85)
axes[0].set_xticks(list(x))
axes[0].set_xticklabels(metric_names, fontsize=11)
axes[0].set_ylim(0, 1.0)
axes[0].set_ylabel('Score')
axes[0].set_title('Overall Metrics')
axes[0].legend()
axes[0].grid(axis='y', alpha=0.3)
for bar in bars1:
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
        f'{bar.get_height():.3f}', ha='center', va='bottom', fontsize=9)
for bar in bars2:
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
        f'{bar.get_height():.3f}', ha='center', va='bottom', fontsize=9, color='darkred')
# Per-class AP50
ap50_20 = list(metrics_20.box.ap50)
ap50_50 = list(metrics_50.box.ap50)
x2 = range(len(CLASSES))
axes[1].bar([i - width/2 for i in x2], ap50_20, width, Label='Epoch 20', color='steelblue', alpha=0.85)
axes[1].bar([i + width/2 for i in x2], ap50_50, width, Label='Epoch 50', color='coral', alpha=0.85)
axes[1].set_xticks(list(x2))
axes[1].set_xticklabels(CLASSES, rotation=35, ha='right', fontsize=9)
axes[1].set_ylim(0, 1.0)
axes[1].set_ylabel('AP@0.5')
axes[1].set_title('Per-Class AP@0.5')
axes[1].legend()
axes[1].grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('/kaggle/working/epoch_comparisonBars.png', dpi=150, bbox_inches='tight')
plt.show()
# — Figure 2: Training curves side by side —————
fig, axes = plt.subplots(1, 2, figsize=(18, 7))
fig.suptitle('Training Curves – Epoch 20 vs Epoch 50', fontsize=14, fontweight='bold')
for ax, run_dir, label in zip(axes, [RUN_20, RUN_50], ['Epoch 20', 'Epoch 50']):
    p = run_dir / 'results.png'
    if p.exists():
        ax.imshow(mplimg.imread(str(p)))
        ax.set_title(label, fontsize=12, fontweight='bold')
    else:
        ax.text(0.5, 0.5, f'{label}\nresults.png not found',
            ha='center', va='center', transform=ax.transAxes)
    ax.axis('off')
plt.tight_layout()
plt.savefig('/kaggle/working/epoch_comparisonCurves.png', dpi=150, bbox_inches='tight')
plt.show()
# — Figure 3: Sample predictions side by side —————
import cv2
import numpy as np
BASE = Path('/kaggle/input/datasets/banuprasadb/visdrone-dataset')
VISDRONE = BASE / 'VisDrone_Dataset'
VAL_IMG = VISDRONE / 'VisDrone2019-DET-val' / 'images'
sample_imgs = sorted(VAL_IMG.glob('*.jpg'))[:4]
fig, axes = plt.subplots(4, 2, figsize=(18, 22))
fig.suptitle('Sample Predictions – Epoch 20 (left) vs Epoch 50 (right)',
    fontsize=14, fontweight='bold')
for row, img_path in enumerate(sample_imgs):
    img_bgr = cv2.imread(str(img_path))
    for col, (m, label) in enumerate([(model_20, 'Epoch 20'), (model_50, 'Epoch 50')]):
        results = m(img_bgr, conf=0.25, verbose=False)[0]
        person_count = sum(1 for b in results.bboxes if int(b.cls) in {0, 1})
        car_count = sum(1 for b in results.bboxes if int(b.cls) == 3)
        annotated = results.plot()
        annotated_rgb = cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB)
        banner = np.zeros((45, annotated_rgb.shape[1], 3), dtype=np.uint8)
        banner[:] = (25, 25, 25)
        cv2.putText(banner, f'{label} | Persons: {person_count} Cars: {car_count}',
            (10, 30), cv2.FONT_HERSHEY_DUPLEX, 0.75, (255, 255, 255), 2)

```

```

    banner_rgb = cv2.cvtColor(banner, cv2.COLOR_BGR2RGB)
    out = np.vstack([banner_rgb, annotated_rgb])
    axes[row, col].imshow(out)
    axes[row, col].set_title(f'{label} - {img_path.name[:25]}', fontsize=8)
    axes[row, col].axis('off')

plt.tight_layout()
plt.savefig('/kaggle/working/epoch_comparison_predictions.png', dpi=120, bbox_inches='tight')
plt.show()
print("\n✓ All comparison outputs saved:")
print(" /kaggle/working/epoch_comparisonBars.png")
print(" /kaggle/working/epoch_comparisonCurves.png")
print(" /kaggle/working/epoch_comparisonPredictions.png")

```

Output:

```

Evaluating 20-epoch model...
Ultralytics 8.4.51 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Model summary (fused): 73 layers, 11,129,454 parameters, 0 gradients, 28.5 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 213.8±129.2 MB/s, size: 150.7 KB)
val: Scanning /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone Dataset/VisDrone2019-DET-val/labels... 548 images, 0 backgrounds, 0 corrupt: 100% 548/548 813.9i
WARNING ⚠ val: Cache directory /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone Dataset/VisDrone2019-DET-val is not writable, cache not saved.
Class Images Instances Box(P R mAP50 mAP50-95): 100% 35/35 4.0it/s 8.6s0.2s
all 548 38759 0.58 0.298 0.248 0.152

Speed: 0.5ms preprocess, 4.9ms inference, 0.0ms loss, 3.7ms postprocess per image
Results saved to /kaggle/working/runs/detect/val-7
Evaluating 50-epoch model...
Ultralytics 8.4.51 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Model summary (fused): 73 layers, 11,129,454 parameters, 0 gradients, 28.5 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 276.7±104.0 MB/s, size: 164.3 KB)
val: Scanning /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone Dataset/VisDrone2019-DET-val/labels... 548 images, 0 backgrounds, 0 corrupt: 100% 548/548 939.0i
WARNING ⚠ val: Cache directory /kaggle/input/datasets/banuprasadb/visdrone-dataset/VisDrone Dataset/VisDrone2019-DET-val is not writable, cache not saved.
Class Images Instances Box(P R mAP50 mAP50-95): 100% 35/35 4.0it/s 8.9s0.2s
all 548 38759 0.583 0.358 0.299 0.185

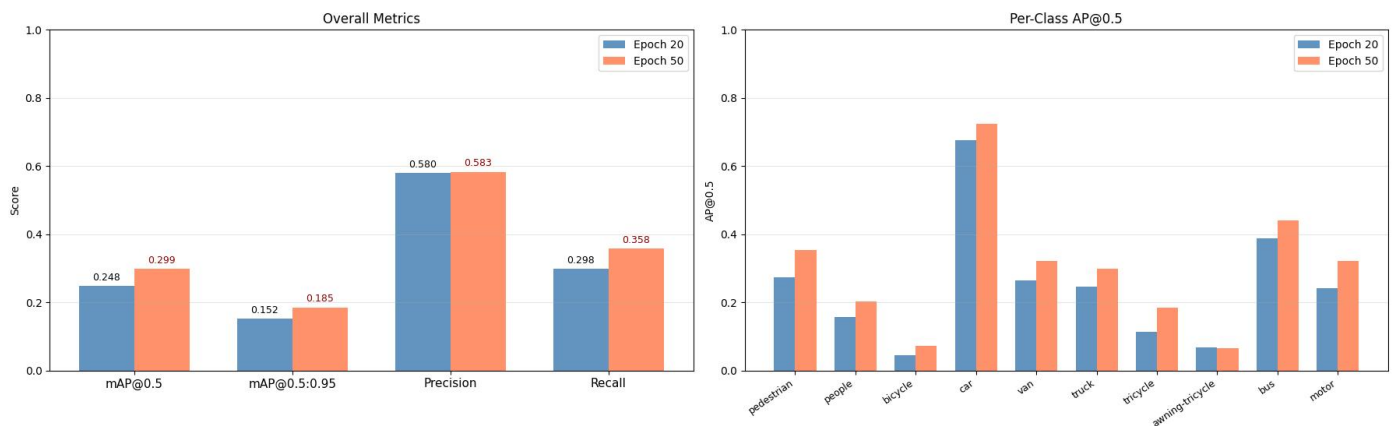
Speed: 0.5ms preprocess, 5.4ms inference, 0.0ms loss, 3.5ms postprocess per image
Results saved to /kaggle/working/runs/detect/val-8

— EPOCH 20 vs EPOCH 50 —
Metric Epoch 20 Epoch 50 Improvement Improvement %
mAP@0.5 0.2475 0.2987 0.0511 +20.66%
mAP@0.5:0.95 0.1520 0.1853 0.0332 +21.87%
...
tricycle 0.1143 0.1841 +0.0698
awning-tricycle 0.0675 0.0660 -0.0016
bus 0.3885 0.4416 +0.0531
motor 0.2412 0.3226 +0.0814

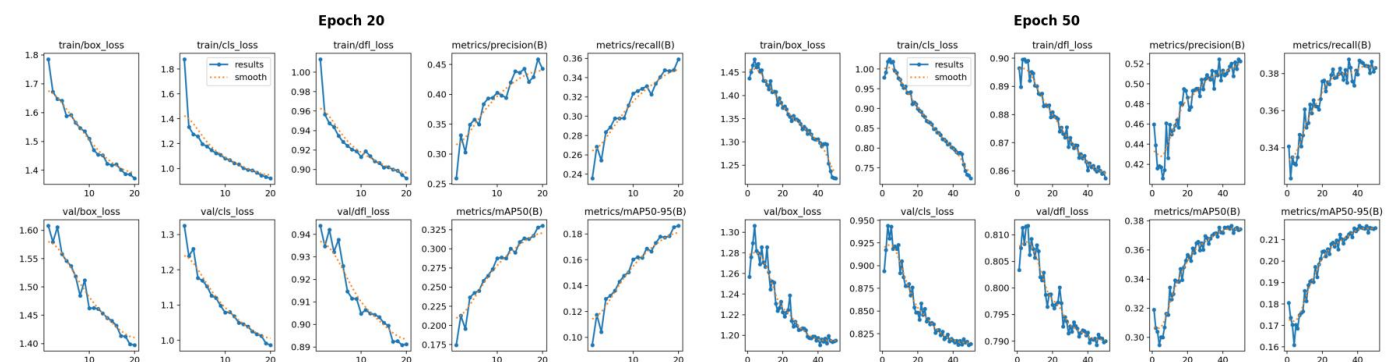
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
/tmp/ipykernel_57/2852827238.py:44: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead.
df[['Epoch 20', 'Epoch 50', 'Improvement']] = df[['Epoch 20', 'Epoch 50', 'Improvement']].applymap(lambda x: f'{x:.4f}')

```

Epoch 20 vs Epoch 50 — Model Comparison



Training Curves — Epoch 20 vs Epoch 50





✓ All comparison outputs saved:
 /kaggle/working/epoch_comparisonBars.png
 /kaggle/working/epoch_comparisonCurves.png
 /kaggle/working/epoch_comparisonPredictions.png